Ders 14 Çalışma Özeti

Ders 14 Çalışma Özeti

Genel Konular

- Dönemin son dersidir. Bir önceki derste sunum yapmayan bütün gruplar dönem projesinin test sunumunu yapar.
- Dokuz grup ve tek kişilik bir proje sunulur. Her sunum yazılımın tanıtımı, test planı, birim testi (unit test), sistem testi ve kullanıcı testi bölümlerinden oluşur.
- Sunulan yazılımlar:
  1. Dil kursu otomasyon sistemi (C# masaüstü uygulaması)
  2. İnsan kaynakları sistemi (Java, JavaFX)
  3. Bilgisayar mühendisliği proje yönetim sistemi (Django)
  4. Seyahat acentesi uygulaması (C# Windows Forms, MSSQL)
  5. Otopark otomasyon sistemi (Java masaüstü)
  6. Sherlock (açık kaynaklı, Python ile yazılmış komut satırı aracı)
  7. Küçük bir alışveriş uygulaması (Flask)
  8. Instagram otomatik hesap oluşturma botu (tarayıcı eklentisi)
  9. Android TV için Leanback Launcher arayüzü
  10. Tek kişilik depo stok yönetim sistemi
- Ders sonunda ek ders, sınav haftası ve final sınavının biçimi duyurulur.

Hocanın Özellikle Vurguladığı Kısımlar

- Her grubun süresi 15-20 dakikadır. Sunum, gruptaki herkes bir bölüm anlatacak şekilde paylaştırılmalıdır.
- Proje tanımı hızlı geçilmelidir.
- Gereksinim analizinde yalnızca fonksiyonel ve fonksiyonel olmayan özellikler kısaca sıralanmalıdır. Use case diyagramları ayrıntılı gösterilmemeli, senaryolardan birkaç örnek verilmelidir.
- **Güvenilirlik (reliability) ile güvenlik (security) ayrı kavramlardır.**
  - 1. grup "güvenilirlik gereksinimi" başlığı altında kimlik doğrulamayı anlatır.
  - Hoca bunun bir kavram hatası olduğunu, anlatılanın güvenlik olduğunu söyler.
- Hoca birim testi ve sistem testi için hangi aracın (tool) kullanıldığını her gruptan açıkça duymak ister. Araç sorulduğunda ürünün değil, test aracının adı verilmelidir.
- Sistem test sonuçları sunuma sığmıyorsa raporda verilebilir. Hoca bunlara rapordan bakacaktır.
- Hoca 5. grubun kapsam (coverage) ölçümü için bir araç kullanmasını özellikle olumlu bulur.
- Tek kişilik projede hoca birim ve sistem testi adımlarını takip edemediğini söyler. Sunumun genel kaldığını, yalnızca kullanıcı testinin gösterildiğini belirtir.
- Hoca, üyeleri projeyi bırakan iki kişilik gruba ve tek kişilik projeye yapabildikleri kadarını tamamladıkları için teşekkür eder.

Kısa Tekrar Notları

- Test seviyeleri birbirini tamamlar. 4. grupta birim testlerinin hepsi geçer, ama kullanıcı testleri 9 hata bulur. Hatalar birimlerde değil, modüllerin birleşiminde çıkar.
- Giriş kriteri (entry criteria): testin başlaması için sağlanması gereken koşullardır. Örnek: gerekli alanlar doldurulmuş, kod yazılmış ve gözden geçirilmiş olmalıdır.

- Çıkış kriteri (exit criteria): testin bitmiş sayılması için gereken koşullardır. Örnekler: testlerin en az %80'i başarılı, hedef kalite seviyesi %95.
- Grupların ortak hata derecelendirmesi:
  - 1. derece hata: sistemin genel çalışmasını doğrudan etkiler.
  - 2. derece hata: sistemi durdurmaz ama bir işlevi engeller.
  - 3. derece hata: görsel bir kusurdur.
- Kullanıcı testinde use case senaryoları test edilir: ana başarı senaryosu ve alternatif senaryolar. Bunlara ek olarak senaryoda olmayan, sıradan bir kullanıcının deneyebileceği davranışlar da test edilebilir.
- Alfa testi geliştirici ekip tarafından yapılır. Beta testi gerçek son kullanıcılarla yapılır.
- Gruplarda kullanılan araçlar:

Amaç	Araçlar
Birim testi	NUnit, MSTest, JUnit, Django `TestCase`, Python `unittest`
Yük ve performans	Apache JMeter, Locust, Windows Performance Monitor, Visual Studio Performance Profiler, PyCharm Profiler
Masaüstü arayüzde sistem testi	Winium, WinAppDriver
Web arayüzünde sistem testi	Selenium, Katalon Recorder
Android arayüzünde sistem testi	TestProject (Appium tabanlı), Selendroid
Kapsam (coverage)	IntelliJ IDEA, PyCharm

Detaylı Açıklamalar

1. grup: Dil kursu otomasyon sistemi

- **Yazılım:** C# ile yazılmış bir masaüstü uygulamasıdır. Şube, öğrenci, öğretmen, ders ve sınıf kayıtlarını tek programda tutar. Yönetici ve şube görevlisi olmak üzere iki rol vardır.
- **Gereksinim örneği (öğrenci silme):**
  - Ön koşul: yönetici veya şube görevlisi sisteme giriş yapmış olmalıdır.
  - Son koşul: öğrenci sistemden silinmiş olur.
  - Alternatif senaryo: silinmek istenen öğrenci sistemde yoksa "böyle bir öğrenci yok" uyarısı çıkar.
- **Fonksiyonel olmayan gereksinimler:** kullanıcı dostu arayüz, işlemlerin 10 saniyeden kısa sürmesi, yedekleme ve yedekten geri yükleme, ad ve şifreyle kimlik doğrulama.
- **Test planı:**
  - Yazılım kullanıcı girdisine çok bağlıdır. Bu yüzden fonksiyonel gereksinimler farklı girdi kombinasyonlarıyla denenir. Örnek: öğretmen eklenirken saat seçilmiş ama şube seçilmemiş.
  - Beklenen hata mesajı veriliyorsa test başarılı sayılır. Beklenmeyen bir hata çıkarsa test başarısızdır.
  - Test sonucu hem gereksinimlere hem kaynak koda bakılarak değerlendirilir.
  - Test ortamı Visual Studio, birim test aracı NUnit'tir.
- **Birim testi:** 14 test çalıştırılır, 12'si geçer, 2'si başarısız olur:
  - Geçersiz öğrenci ID'siyle aramada veritabanı komutunun boş olması beklenir. Ancak komut yine de oluşturulur.
  - Ev ve cep telefonu alanına sayısal olmayan değer girilince hata beklenir. Ancak bu kontrol yapılmaz.
- **Sistem testi:** Uygulama masaüstü olduğu için Windows Performance Monitor kullanılır. CPU yükü, disk erişimi ve thread sayıları düşük çıkar.
- **Kullanıcı testi:**
  - Önceden belirlenmiş kriterlerle bir kullanıcı anketi hazırlanır. Kullanıcı her özelliği olumlu, olumsuz veya nötr değerlendirir.
  - Gereksinim bazında da test yapılır: her işlemin önceliği, kullanıcının değerlendirmesi ve yorumu kaydedilir.
- **Güvenlik:** Şube görevlisi ders ve sınıf ekleyemez. Bu işlemler yalnızca yöneticiye açıktır.

2. grup: İnsan kaynakları sistemi

- **Yazılım:** Çalışan bilgilerini tutar, çalışanları projelere yerleştirir. Maaş, tazminat ve sigorta hesaplarını yapar.
- **Altyapı:** Java (IntelliJ IDEA), MySQL (localhost), JavaFX, Scene Builder, CSS ve FXML. Testler JUnit ile yazılır, derleme Maven ile yapılır.
- **Gereksinimler:** 40 fonksiyonel ve 32 fonksiyonel olmayan gereksinim belirlenir. Fonksiyonel olmayanların 13'ü performans, 13'ü kullanılabilirlik, 6'sı güvenlikle ilgilidir.
- **Test planı:**
  - Test edilen modüller: çalışan, proje ve takım.
  - Tamamlanma koşulları: planlanan testlerin hepsi yapılmış, tüm test durumları başarıyla sonuçlanmış ve 1., 2. ve 3. derece hataların tamamı çözülmüş olmalıdır. Tüm test faaliyetleri raporlanmış olmalıdır.
- **Birim testi:** toplam 56 JUnit testi yazılır. Testler şunları kapsar:
  - veritabanı bağlantısı
  - kullanıcı kaydı
  - yetkili ve yetkisiz giriş
  - takım çekme ve uyarı testleri
  - proje kontrolü
  - Canlı gösterimde tüm testler çalıştırılınca 3 testin başarısız olduğu görülür.
- **Sistem testi:**
  - Performans testi, yük testi ve güvenlik testi Apache JMeter ile yapılır. Sonuçlar zaman kısıtı nedeniyle yalnızca raporda verilir.
  - Performans testinde istenen değerler içinde desteklenebilecek en fazla kullanıcı sayısı bulunur. Yük testinde belirli bir yük altındaki tepki ölçülür.
- **Kullanıcı testi:**
  - Veritabanı yokken sistem uyarı veriyor mu?
  - Yeni çalışan eklenince sistem otomatik olarak uygun bir proje buluyor mu? Örnek: yazılımcı eklenince web sitesi geliştirme projesi önerilir. Bir takımda en fazla 5 çalışan olabilir.

3. grup: Bilgisayar mühendisliği proje yönetim sistemi

- **Yazılım:**
  - Bitirme ve ara projelerinde öğrenci-hoca iletişimi her hocaya göre farklı kanallardan yürür. Bu sistem bunu tek yerde toplamayı amaçlar.
  - Öğrenci proje önerisini sisteme yükler. Akademisyenler ve asistanlar önerileri görüp onaylar.
  - Ara rapor, bitirme raporu ve sunum gibi dosyalar da buradan ilerletilir.
- **Birim testi:** Django'nun kendi `TestCase` sınıfıyla 9 test yazılır, hepsi geçer:
  - Giriş:
    - Doğru bilgilerle POST isteği atılır. Durum kodunun başarılı olduğu ve ana sayfaya yönlendirme yapıldığı kontrol edilir.
    - Oturum boyunca aktif kullanıcının bağlamda (context) bulunduğu kontrol edilir.
    - Alan boş bırakılınca form hatası verildiği kontrol edilir.
  - Duyurular:
    - Örnek duyurular oluşturulur. Ana sayfanın bunları içerdiği `contains` kontrolüyle doğrulanır.
    - Öğrenci rolü duyuru oluşturma sayfasına girmek isteyince giriş sayfasına yönlendirilir. Koordinatör ise sayfaya erişebilir.
  - Proje önerisi:
    - Akademisyen bir öneri oluşturur. Öneri sayfasına yönlendirme yapıldığı kontrol edilir.
    - Onaylanmamış öneri ana sayfada listelenmemelidir. Bu, `assertNotContains` ile kontrol edilir.
    - Önerinin durumu "accepted" yapılınca öneri listede görünmelidir. Bu kez `assertContains` kullanılır.
- **Performans testi:**
  - SRS'teki 36 numaralı gereksinim, sistemin eş zamanlı kullanıcı yükünü kaldırmasını ister.

  - Açık kaynaklı Locust aracıyla 50 sanal kullanıcı aynı anda ana sayfaya istek gönderir. Bu, saniyede 8-9 isteğe karşılık gelir.
  - Grup aynı anda en fazla 50 kişinin gireceğini tahmin eder. Test başarılı olur.

4. grup: Seyahat acentesi uygulaması

- **Yazılım:**
  - Grup bir önceki yılın yazılım mühendisliği dersinde yaptığı projeyi seçmiştir.
  - Uygulama C# Windows Forms ile yazılmış, MSSQL kullanan yerel bir sistemdir.
  - Sistem yöneticisi kullanıcı, firma, tatil, ödeme tipi, yolculuk tipi ve sefer ekler. Müşteri temsilcisi bilet satar, tatil rezervasyonu yapar ve müşteri ekler.
- **Test ortamı:** birim testi için MSTest, performans için Visual Studio. Test planı ve raporu için Excel ve Word kullanılır.
- **Test takvimi:** Ekim ayında test spesifikasyonu ve SRS hazırlanır. Ardından sırasıyla test planı, test senaryoları, testlerin yürütülmesi ve test raporu gelir.
- **Birim testi:**
  - 12 birim testi yazılır. Alt durumlarla birlikte yaklaşık 30 test olur ve hepsi geçer.
  - Butonlar `private` tanımlı olduğu için test edilemez. Grup buton işlemlerini `public` fonksiyonlara taşır ve testleri bu fonksiyonlar üzerinden yazar.
  - Testlerde veritabanı da kullanılır. İnternette "testte gerçek veritabanına bağlanmayın" önerileri vardır. Grup, veritabanında önemli bilgi olmadığı için bunu kabul edilebilir bulur.
  - Örnekler: kullanıcı ekleme, kullanıcı düzenleme (3 test, boş alan kontrolü dâhil) ve kullanıcı silme. Sonuçlar veritabanı tablosundan da kontrol edilir.
- **Veritabanı tasarım hataları:**
  - Firmalar tablosunda hizmet tipi ID'si için yabancı anahtar (foreign key) tanımlanmamıştır.
  - Tetikleyici (trigger) kullanılmadığı için bir hizmet tipi silinince ona bağlı firma kayıtları silinmez. Bu da tutarsızlığa yol açar.
- **Kullanıcı testi:**
  - Testler iki gruba ayrılır: use case senaryolarındaki testler ve senaryoda olmayan, sıradan kullanıcı davranışları.
  - Bilet satışı, sefer ekleme, kullanıcı düzenleme, müşteri ekleme, firma düzenleme ve ekleme gibi modüller ana ve alternatif senaryolarda %100 başarılıdır.
  - Tatil ekleme ve tatil rezervasyonu modülleri hiç çalışmaz. Başarı %0'dır ve hata 1. derece (kritik) sayılır.
  - Senaryo dışı testlerin 5'inin hepsi başarısız olur:
    * Firma veya kullanıcı seçimi yapılmadan işlem yapılınca `null` kontrolü olmadığı için hata çıkar.
    * Yanlışlıkla eklenen bir ödeme tipi silinemez, çünkü silme modülü yoktur.
    * Sıfır koltuklu bir yolculuk tipi kabul edilir. Bu 3. derece (minör) bir hatadır.
    * Kalkış saati geçmiş bir sefere bilet satılabilir. Bu 2. derece bir mantık hatasıdır.
- **Performans testi:**
  - Uygulama yereldir ve tek kullanıcıya göre tasarlanmıştır. Grubun 5 üyesi bir iş günü boyunca uygulamayı kullanır: biri sistem yöneticisi, dördü satış temsilcisi rolündedir.
  - Uygulama release modunda çalıştırılır. Visual Studio Performance Profiler ile CPU ve bellek izlenir.
  - Ortalama bellek kullanımı yaklaşık 40 MB'tır. CPU'yu en çok arayüz oluşturma ve veritabanı sorguları kullanır.
- **Sonuç:**
  - Toplam 75 test yapılır: 30 birim, 40 kullanıcı ve 5 performans testi. Birim testlerinin hepsi geçer, kullanıcı testlerinde 9 hata bulunur.
  - Grubun yorumu: Atomik işlemler mantıksal olarak doğru çalışır. Asıl sorun entegrasyondadır: bazı modüller hiç yazılmamış, bazılarında `null` kontrolü yoktur.

5. grup: Otopark otomasyon sistemi

- **Yazılım:** Java ile yazılmış bir masaüstü uygulamasıdır. Araç girişi ve çıkışı, üyelik kaydı, park durumu sorgulama, saat bazlı ücretlendirme, kapasite, boş yer ve güncel kazanç bilgilerini yönetir. Araç tipi resmî, normal veya otoparka kayıtlı olabilir.

- **Test planı:**
  - Test edilen öğeler:
    - Fonksiyonel gereksinimler: araç girişi, araç çıkışı, üyelik kaydı.
    - Fonksiyonel olmayan gereksinim: güvenlik.
  - Test edilmeyen öğeler: arayüz bilgilendirmeleri, performans, platform ve tasarım.
  - Testlerin amaçları:
    - Kullanıcı gereksinimlerinin karşılandığından emin olmak.
    - Hataları kullanıcıya ulaşmadan düzeltmek.
    - Ürün piyasaya çıktıktan sonra ortaya çıkabilecek yüksek maliyetli hataları önlemek.
  - Roller: uygulama geliştirici, test yöneticisi ve müşteri.
  - Şelale (waterfall) yaklaşımı izlenir: önce tüm gereksinimler gerçekleştirilir, sonra teste geçilir. Modül seviyesinde birim testi, uygulama seviyesinde sistem ve kullanıcı testi yapılır.
  - Tamamlanma kriterleri: tüm testler çalıştırılmış olmalı, bulunan hatalar bir sonraki teste kadar düzeltilmeli ve testlerin en az %80'i başarılı olmalıdır.
  - Doğrulama: fonksiyonel gereksinimler testlerle, fonksiyonel olmayanlar gözlemle doğrulanır.
- **Risk yönetimi:** dört risk vardır:

Risk	Olasılık	Etki	Açıklama
Proje süresi	Yüksek	Büyük	İşlerin gecikmesi projenin bitişini uzatır.
Test ekibi	Orta	Büyük	Ekip başka projelerde çalışırsa testler aksar. Çözüm: iş planı.
Araçlar	Orta	Büyük	Test araçlarının yapılandırılması gerekir.
Test süresi	Orta	Büyük	Modüller zamanında kodlanmazsa testler zamanında bitmez.

- **Birim testi:** JUnit ile yaklaşık 35 test yazılır, hepsi geçer.
  - Test edilen işlemler: geçerli ve geçersiz verilerle araç girişi, araç çıkışı, park durumu sorgulama, üyelik kaydı ve kapasite güncellemesi.
  - Aynı plakalı araç ikinci kez eklenmemelidir.
  - Grup, JetBrains'in bir anketine göre Java geliştiricilerinin büyük çoğunluğunun JUnit kullandığını aktarır.
- **Testler sırasında fark edilen hatalar:**
  - Plaka giriş alanı SQL injection saldırısına açıktır. Kullanıcı hasılat bilgisini değiştirebilir veya tabloları silebilir.
  - Güncel tarihten önceki bir tarihle araç girişi yapılabilir.
- **Sistem testi:**
  - Winium kullanılır. Winium, Windows masaüstü uygulamaları için Selenium tabanlı bir araçtır.
  - Araç girişi ve araç çıkışı senaryoları otomatik çalıştırılır ve beklenen sonuçlar alınır.
  - Uçtan uca (end-to-end) test canlı olarak da gösterilir.
- **Kapsam testi:** IntelliJ IDEA'nın yerleşik aracıyla ölçülür, sonuç %20'dir. Grup bunu kodda testlerin çalıştırmadığı çok sayıda arayüz kodu bulunmasına bağlar.
- **Kullanıcı testi:** son kullanıcı araç girişini dener ve sorun çıkmaz.

6. grup: Sherlock

- **Yazılım:** Python ile yazılmış, komut satırından çalışan açık kaynaklı bir araçtır. Verilen kullanıcı adını yüzlerce sosyal medya sitesinde arar ve bulunan profil bağlantılarını listeler.

  - GitHub'da 15-20 bin yıldızı olan, yıllardır geliştirilen bir projedir. Bu yüzden testlerde çok az hata bulunur.
- **Gereksinim analizi:** Siteler bir kullanıcının var olup olmadığını üç farklı yolla bildirir:
  - HTTP durum kodu
  - yanıt URL'si (response URL)
  - hata mesajı
  - Grup bu yüzden her testi bu üç mekanizma için ayrı ayrı yazar.
  - Fonksiyonel olmayan gereksinim: yüzlerce siteden veri sırayla değil, eş zamanlı çekilmelidir.
- **Test planı:**
  - Doğru ve hatalı kullanıcı adı girdileri test edilir.
  - Üç mekanizmadan gelen sonuçlar ayrı ayrı kontrol edilir.
  - Proxy ve Tor üzerinden bağlantı kurulup kurulamadığı test edilir.
  - Çıktı raporları test edilir.
  - Test ortamı: Python 3, internet bağlantısı ve kurulu Tor. Kod PyCharm'da çalıştırılır.
  - Araçlar: birim testi için Python `unittest`, sistem testi için Microsoft WinAppDriver, performans için PyCharm Profiler.
  - Test çıktıları: test planı, test durumları, test kayıtları (log) ve test raporu.
- **Birim testi:** 13 test yazılır, hepsi geçer. Testler önceden hazırlanmış test verisini okur ve siteleri sırayla dener.
  - Geçerli kullanıcı adı aramayı tetikliyor mu?
  - Parantez veya kesme işareti gibi geçersiz karakterli ad hata mesajı veriyor mu?
  - Durum kodu ve yanıt URL'si kullanan sitelerde sonuç doğru raporlanıyor mu?
- **Sistem testi:** Proxy ve Tor gibi aracı sistemlerle bağlantının doğru çalışıp çalışmadığı denenir.
- **Performans ve kapsam:**
  - PyCharm Profiler fonksiyonların çalışma sürelerini gösterir. Kabul edilebilir süreler yeşil, uzun süreler kırmızıdır.
  - Eş zamanlılık testinde 19 farklı thread görülür.
  - Toplam kapsam yaklaşık %70'tir.

7. grup: Alışveriş uygulaması

- **Durum:** Grubun üç üyesi görev dağılımından sonra projeyi bırakmıştır. Kalan iki kişi yalnızca birim testi ve kullanıcı kabul testi (user acceptance test, UAT) yapabilir. İlk seçilen uygulama ayağa kaldırılamadığı için benzer bir uygulamaya geçilmiştir.
- **Yazılım:** Flask ile yazılmış küçük bir alışveriş uygulamasıdır. Giriş, kayıt, profil güncelleme, şifre değiştirme, sepete ürün ekleme ve çıkarma işlemlerini destekler. Ödeme özelliği yoktur.
- **Kullanıcı kabul testi:**
  - Alfa testini iki geliştirici yapar. Beta testini son kullanıcı olarak seçilen iki tanıdık yapar.
  - Ortamlar Windows 10 ve macOS'tur.
  - Uygulama küçük olduğu için bütün işlevler kullanıcı üzerinde test edilir ve testler geçer.
  - Riskler: zaman kısıtı yüzünden tamamlanamayan testler, deneyimsiz test ekibi, yetersiz use case ve hata giderimi.
- **Birim testi:**
  - Kayıt:
    - Geçersiz e-posta biçimi ve boş parola reddedilir.
    - Aynı bilgilerle ikinci kez kayıt olunabilir. Bu, uygulamada bulunan bir kusurdur.
  - Giriş:
    - Doğru e-posta ve parolayla giriş yapılır.
    - Yanlış parola veya yanlış e-postayla giriş yapılamaz.
  - Profil: giriş yapmadan profil sayfasına gidilince ana sayfaya yönlendirilir.
  - Şifre değiştirme: eski parola yanlışsa parola değişmez.
  - Ürünler ve sepet:
    - Ürün sayfaları görüntülenir.
    - Sepet yalnızca giriş yapan kullanıcıya açıktır.
    - Eklenen ürün sepette görünür.

8. grup: Instagram otomatik hesap oluşturma botu

- **Yazılım:** Chrome ve Firefox için bir tarayıcı eklentisidir. Geçici (fake) bir e-posta alır, ad-soyad, kullanıcı adı, şifre ve doğum tarihini doldurur. Gelen doğrulama koduyla kaydı tamamlar.
- **Test planı:**
  - Birim testi modülleri: e-posta doğrulama, ad-soyad doğrulama, kullanıcı doğrulama ve şifre doğrulama.
  - Performans testleri: yük, stres, dayanıklılık, ani yük (spike), hacim ve ölçeklenebilirlik.
  - Araçlar: birim testi için Python standart kütüphanesi (`unittest`), sistem testi için Selenium, performans için JMeter.
  - Hatalar derecelendirilir. Risk yönetiminde sistemden ve ekipten gelen riskler ele alınır.
- **Birim testi:** Bot dış servislere bağlı olduğu için çok az fonksiyon birim olarak test edilebilir. Test edilenler:
  - Bot tarafından üretilen e-posta, ad-soyad ve şifre Instagram'ın giriş alanına aynen yazılıyor mu?
  - Eşleşme varsa test "OK", yoksa "fail" döner ve hatanın yerini gösterir.
- **Sistem testi:**
  - Uygulama yerelde çalıştığı ve arkada ağır bir sistem olmadığı için yalnızca yük testi yapılır. Test, Instagram adresine erişilip erişilemediğini kontrol eder.
  - Tarayıcı adımları Katalon Recorder ile kaydedilir ve dışa aktarılır. Oluşan test örneği JMeter'da çalıştırılır.

9. grup: Leanback Launcher (Android TV arayüzü)

- **Yazılım:**
  - Android TV cihazları için bir başlatıcı (launcher) arayüzüdür. Uygulamaları gruplar, favori ekleme ve silmeye izin verir.
  - Kumandadaki kırmızı, yeşil, sarı ve mavi tuşlara uygulama atanabilir.
  - Dil, tarih-saat, ekran, ağ ve Bluetooth ayarlarına hızlı erişim sağlar. Cihaz bilgisini gösterir.
  - Ek olarak, açık uygulamaların tuttuğu belleği serbest bırakan bir "Clear Memory" uygulaması yazılmıştır.
- **Test planı:**
  - Test edilen modüller: favori uygulama ekleme, uygulama silme ve uygulama ekleme.
  - Giriş kriterleri: kodun yazılması ve gözden geçirilmesi, birim testlerinin yapılması.
  - Çıkış kriterleri: tüm kabul kriterlerinin sağlanması ve %95 hedef kalite seviyesine ulaşılması.
  - Hata yönetimi Git platformunda yapılır. Hata derecesine göre etiket verilir: 1. derece "acil", 2. derece "önemli".
  - Riskler: geliştirme veya test ekibinin başka projelerde çalışması, modüllerin zamanında bitmemesi ve test ortamının yetersizliği.
- **Araçlar:** birim testi için JUnit. Sistem testi için TestProject (Appium tabanlı) ve Selendroid kullanılır. Testler bir Android TV kutusunda ve bir telefonda çalıştırılır.
- **Sistem testi:**
  - Test senaryosu adım adım kaydedilir. Araç adımları kendisi uygular ve sonunda bir koşulu kontrol eder. Örnek: Chrome favorilere eklendi mi?
  - Aynı test 50-60 kez tekrarlanır. Kaç kez geçtiği ölçülür.
  - İlk ölçümde favori ekleme testi yaklaşık 92 çalıştırmada %74 başarı verir.
  - İnceleme sonunda neden bulunur: arayüz, liste yüklenmeden yukarı kaymaya çalışmaktadır. Düzeltmeden sonra 42 çalıştırmanın hepsi başarılı olur.
  - Araç, başarısız adımın ekran görüntüsünü rapora ekler. Aynı test birden fazla cihazda aynı anda çalıştırılabilir.
  - Canlı gösterimde başarı %71 çıkar. Grup bunu, gösterim sırasında cihaz bağlantısının kesilmesine bağlar.
- **Performans:**
  - Açılışta paket adları, ikonlar ve banner'lar okunduğu için CPU kısa süre yükselir, sonra düşüp sabitlenir.

  - Bellek kullanımı en fazla yaklaşık 64 MB'tır.
  - Düşük sürümlü cihazlarda düşük FPS görülebilir.

10. Tek kişilik proje: Depo stok yönetim sistemi

- **Yazılım:** Ürün ekleme ve silme, stok girişi, güncelleme ve silme işlemlerini yapar. Raporları dışarı aktarabilir. Proje analiz, tasarım, gerçekleştirme, test ve bakım adımlarına bölünmüştür. Ticari bir depo yönetim yazılımı örnek alınmıştır.
- **Testler:**
  - Kullanıcı testinde 5 arayüz ekranı gereksinimlere göre incelenir ve butonların çalışması kontrol edilir.
  - Birim testi Visual Studio'nun birim test aracıyla ve kendi yazılan doğrulama (validation) metotlarıyla yapılır. İki birim testi vardır.
  - Sistem testi olarak iki farklı bilgisayarda işlemci ve bellek kullanımı izlenir.
  - Kullanıcı testinde bir hata bulunur. Hatalar üç dereceye ayrılır. 1. derece hatalar anında çözülür.

Duyurular

- Erasmus öğrencilerinin bulunduğu bir grubun bugün mazereti vardır. Hoca bu grup için ertesi akşam saat 17.00 veya 18.00'de ek ders tanımlayacaktır. Diğer öğrenciler bu ek derse katılmak zorunda değildir, ama davet herkese gidebilir.
- Önümüzdeki hafta sınav haftasıdır.
- Bir öğrenci final sınavının test (çoktan seçmeli) olup olmayacağını sorar. Hoca "büyük ihtimalle test olacak" der. Kesin biçim sınav yüklendiğinde görülecektir.

Ders sonrası notlar

- NUnit, Visual Studio'nun kendi aracı değildir. Visual Studio'ya bir test bağdaştırıcısıyla (test adapter) eklenen açık kaynaklı bir .NET test çatısıdır. Microsoft'un kendi test çatısı MSTest'tir. 4. grup MSTest'i kullanmıştır.
- 1. grup, Visual Studio geniş kapsamlı olduğu için test ve geliştirme ortamının aynı olmasını sorun görmez. Ancak ortam ayrımı IDE ile ilgili değildir. Konu veri, yapılandırma ve kurulumun ayrı tutulmasıdır. Test, geliştiricinin makinesinden bağımsız ve tekrarlanabilir bir ortamda çalışmalıdır.
- 2. grup masaüstü uygulamasının açılış süresini, bellek ve CPU kullanımını JMeter ile ölçtüğünü anlatır. JMeter esas olarak HTTP, JDBC gibi protokoller üzerinden sunucu tarafına yük bindirmek için kullanılır. Bir masaüstü uygulamasının CPU ve bellek kullanımı VisualVM gibi bir profil aracıyla daha doğru ölçülür.
- 2. grubun canlı gösteriminde test sayıları altyazıda karışık geçer ("3 başarısız", "43 başarısız"). 56 testten 3'ünün başarısız olduğu anlaşılmaktadır.
- 4. grubun bulgusu test seviyelerinin önemini gösterir. Bütün birim testleri geçse de modüller birleşince hata çıkabilir. Bu tür hatalar entegrasyon, sistem ve kullanıcı testlerinde ortaya çıkar.
- 4. grup `private` butonları `public` yapar. Daha iyi bir çözüm, iş mantığını arayüzden ayrı bir sınıfa taşımak ve o sınıfı test etmektir. Arayüz kodu test için değiştirilmemiş olur.
- 4. grup testlerde gerçek veritabanını kullanır. Testlerin tekrarlanabilir olması için ayrı bir test veritabanı veya sahte nesneler (mock) tercih edilir. Her test kendi verisini hazırlayıp sonunda temizlemelidir.
- 5. grubun bulduğu SQL injection açığı, kullanıcı girdisinin sorgu metnine doğrudan eklenmesinden kaynaklanır. Çözüm parametreli sorgudur (Java'da `PreparedStatement`).
- Kapsam (coverage) ayrı bir test türü değildir. Mevcut testlerin kodun ne kadarını çalıştırdığını gösteren bir ölçüttür.
- 6. grup sistem testi için Microsoft WinAppDriver'ı anar. WinAppDriver Windows grafik arayüzlü uygulamaları otomatikleştirmek içindir. Komut satırı aracı için sistem testi genelde programı betikle çalıştırıp çıktısını kontrol ederek yapılır.
- 8. grup "Selenium performans veya sistem testi için uygun değil" der. Selenium arayüz üzerinden fonksiyonel ve sistem testi için uygundur. Yük ve performans testi için uygun değildir. Bu iş için JMeter gibi araçlar kullanılır.

- 8. grubun projesi gibi otomatik hesap açan botlar Instagram'ın kullanım koşullarına aykırıdır. Gerçek bir projede bu tür yasal sınırlar da gereksinim ve risk analizine girmelidir.
- 9. grubun yaşadığı durum kararsız test (flaky test) örneğidir. Kararsız test, kod değişmeden bazen geçen, bazen kalan testtir. Sebep çoğu zaman zamanlamadır. Burada liste yüklenmeden işlem yapılmaktadır. Çözüm, sabit beklemeler yerine koşula bağlı (explicit) beklemelerdir.
- Tek kişilik projede sistem testi yalnızca performans ölçümü olarak anlatılır. Sistem testi daha geniştir: bütünleşik sistemin fonksiyonel ve fonksiyonel olmayan gereksinimlerini uçtan uca doğrular.
- Altyazıda bazı adlar bozuk geçer. Doğruları:
  - "Limbeck Launcher" Leanback Launcher
  - "Vinyum" Winium
  - "G-metre / G-Metri" JMeter
  - "katalona rekorder" Katalon Recorder
  - "test projekti / Test Project ION" TestProject.io
  - "Sel Android" Selendroid
  - "Paycham / Payçam" PyCharm
  - "IntelliCity" IntelliJ IDEA
  - "SIM Builder" Scene Builder
  - "Maben" Maven
  - "bulletin test case" Django'nun yerleşik `TestCase` sınıfı

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.